A3 – Securing the access to the Linux webserver

Competency	Points
The trainee is able to secure the network communication of the webserver under Linux.	16

Requirements:

- VM from previous Assignment2 with installation of OpenSSH
- Host windows OS (with WinSCP or Filezilla installed)

Task 1: Transfer of a Webpage to the Webserver

- Create a Website on the Windows OS.
- Transfer this website using **SFTP** to the Linux server and prove that it is running.
- Reuse the MySQL-Database from Chapter 2:
    a. Export your database from your Linux-Server
    b. Import it again
    (Hints: use mysqldump and phpMyAdmin/Adminer)

Task 2: Different User Accounts

1. **Create User Accounts**
   Start by creating user accounts for each role:
   - **apache_admin**: Full access to all server files, directories, and administrative tools.
   - **webserver_IAM**: Access to web content directories for managing websites but no access to database files or critical system functions.
     Note replace IAM by your own and write everything it in small letters!
   - **database_IAM**: Access restricted to database-related directories without access to web content.
     Note replace IAM by your own and write everything it in small letters!
   - **User_IAM**: Read-only access to view web content but no permissions to modify files or directories.
     Note replace IAM by your own and write everything it in small letters!

   Each of these accounts will represent a different role with varying access levels to the server and its resources.

2. **Create User Groups for Roles**
   Next, create distinct groups corresponding to each role (e.g., an admin group, webserver group, database group, and general user group). These groups will be used to assign specific permissions to the users.

3. **Assign Users to Groups**
   Add each user to the appropriate group based on their role. For example, the ApacheAdmin user should belong to the admin group, the webserverIAM user to the webserver group, and so on. This ensures that permissions can be controlled more easily at the group level.

- **Set Permissions for Web Directories**
  a. Configure permissions for the web server's directories.
  b. The webserver user should have full access to edit and manage the web content in the server's web directory (usually located in /var/www/html).
  c. The database user should only have access to directories where database-related files are stored (usually in /var/db_data).
  d. The admin user should have full control over both web and database directories to perform administrative tasks.
  e. The general user should only have read access to the web content but no ability to modify or edit files.
- **Grant Administrative Privileges to the Admin User**
  Ensure that the admin user has elevated privileges. This user should have the ability to perform system administration tasks, such as starting or stopping the web server, managing users, and editing critical server configuration files.
- **Configure File Permissions for .htaccess**
  The .htaccess file, which allows for directory-level configuration of Apache settings, should only be editable by the admin and webserver users. Ensure that other users, such as the database user or the general user, cannot modify this file for security reasons.
- **Restrict Access to Certain Directories Based on Roles**
  Modify the web server configuration so that access to specific directories is controlled based on user roles. For instance, the admin user should be able to access and modify all files, while the webserver user should only have access to web content directories. The database user should only access database-related directories, and the general user should be restricted to viewing content without editing privileges.
- **Restart the Web Server**
  After configuring users, groups, and permissions, restart the web server (Apache) to ensure that all changes take effect. This step ensures the new access controls are applied correctly.

Task 3: PHP Access to Database

1. Create a PHP File to Display Data from the Database

- In your web server's root directory (usually /var/www/html), create a PHP file (e.g., display_data.php) that connects to the database created in Assignment 2.
- The PHP file should fetch data from the database and display it in a structured format on a web page. Use a secure connection method (like PDO or mysqli with prepared statements) to avoid SQL injection vulnerabilities.

PHP File Structure:

- Connect to the database using secure credentials (stored in a secure location).

- Run an SQL query to fetch the data.
- Display the data on the webpage in a readable format (e.g., a table).

2. Set Permissions for the PHP File

To ensure the security of the PHP file, configure its permissions properly:

- **Owner**: Assign ownership of the PHP file to the webserver user so that only this user can modify the content.
- **Group**: Assign the webserver_group to the file for other webserver-related tasks, if needed.
- **Permissions**: Set the file permissions to ensure only the webserver user can edit the file, while others (including the general user) can only read it.

3. Secure Permissions for All Web Files

For a secure web application, you need to set appropriate file and directory permissions:

- **Directories**: Ensure that all directories in /var/www/html are only writable by the webserver user, while readable by everyone else.
- **PHP Files**: Set PHP files to be readable by others but writable only by the webserver user. This prevents unauthorized users from modifying PHP scripts, which could lead to security risks.
- **Sensitive Configuration Files**: If there are configuration files (e.g., database credentials), restrict their permissions so that only the admin or webserver users can access them.

4. Ensure Database Security

- Restrict the database access only to the database user.
- Ensure that the PHP script uses secure practices, such as:
    - Using prepared statements to prevent SQL injection.
    - Storing database credentials securely (e.g., in environment variables or a secure configuration file).

5. Modify Apache Configuration to Enhance Security

- Disable directory browsing in the Apache configuration to prevent unauthorized users from seeing the file structure.
- Ensure that .htaccess is properly configured to deny access to sensitive files like .env or configuration files.$

6. Test the Setup

- After setting up the PHP file and modifying permissions, test the webpage to ensure it correctly displays the data from the database.
- Verify that only authorized users (admin and webserver) can modify the web files and database settings.

Task 4: Secure the Webpage

1. Secure Your Website with Username and Password Based on Groups

You will protect your website by implementing basic authentication using usernames and passwords. Access will be granted based on user groups.

Instructions:

- Set up basic authentication using Apache's .htaccess file.
- Create a password file (.htpasswd) where the usernames and passwords will be stored securely.
- Associate user access based on groups (like admin_group, webserver_group, etc.).
- **Proof**: Show a screenshot or logs that demonstrate that the website is secured by username and password, prompting users for authentication.

2. Test Login with Unknown Username and Show it in Log Files

- Attempt to log in with an invalid username.
- Check Apache's log files (usually located in /var/log/apache2/access.log or /var/log/apache2/error.log) to show that the login attempt was recorded.

Instructions:

- Try logging in with an unknown username or incorrect password.
- Open and inspect the log files to show that the failed login attempt is logged.
- **Proof**: Provide a screenshot or log output showing the failed attempt with details like IP address, time, and error.

3. Install OpenSSL and Create a Self-Signed SSL Certificate

You need to secure your website using SSL encryption by creating a self-signed certificate.

Instructions:

- Install OpenSSL on your system.

- Generate a self-signed certificate.
    - The certificate should include the private key and public certificate, typically stored in /etc/ssl/certs and /etc/ssl/private.
- Configure your Apache web server to use this certificate for HTTPS connections.
- **Proof**: Provide screenshots or steps showing the successful creation of the SSL certificate and Apache configuration to use it.

4. Secure Your Website with HTTPS Using Your Self-Signed Certificate

You will now configure your Apache server to serve the website using HTTPS with the self-signed certificate you created.

Instructions:

- Modify your Apache configuration to enable HTTPS on the server.
- Configure the website to be served over HTTPS using the SSL certificate.
- Test the website by accessing it via https://yourdomain.com.
- **Proof**: Show that the website is accessible over HTTPS. Provide a screenshot of the browser showing the SSL padlock and self-signed certificate warning.

5. Automatically Redirect HTTP to HTTPS

To ensure secure communication, configure your website to automatically redirect users from HTTP to HTTPS.

Instructions:

- Modify the Apache configuration or use the .htaccess file to set up automatic redirection from HTTP to HTTPS.
- Test this by accessing the website via http://yourdomain.com, and verify that it redirects to https://yourdomain.com.
- **Proof**: Provide a screenshot showing the redirection from HTTP to HTTPS in the browser's address bar.

Note:

→Document each of your steps.

→Your documentation should include screenshots from your configurations showing your username (student abbreviation) as well and as many explanations as possible.

→Your layout of you documentation should be appealing (table of contents, meaningful headlines, structured).